МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТк

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ИИ26(з)

Выполнил
А.В. Семёнов,
студент группы ИИ-26

Проверил
К.В. Андренко,
ст. преп. кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:

- Заменить пороговую функцию активации на сигмоидную.
- Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
- Реализовать вычисление градиента ŷ-exi для обновления весов.

2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом:

- На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
- Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
- Сохранять значения Esp для построения графика.

4. Провести сравнительный анализ:

    - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
        - MSE + фиксированный шаг (данные из ЛР №1);
        - MSE + адаптивный шаг (данные из ЛР №2);
        - BCE + фиксированный шаг;
        - BCE + адаптивный шаг.
    - Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).

5. Визуализировать результаты:

    - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
    - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
    - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.

6. Реализовать режим функционирования:

- Пользователь вводит координаты x1,x2.
- Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
- Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).

7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.

8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

3.

x_1	x_2	e
3	4	1
-3	4	1
3	-4	0
-3	-4	0

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

X_raw = np.array([[3.0, 4.0], [-3.0, 4.0], [3.0, -4.0], [-3.0, -4.0]], dtype=float)
E = np.array([1.0, 1.0, 0.0, 0.0], dtype=float)

def sigmoid(s):
    if s >= 0:
        z = np.exp(-s)
        return 1.0 / (1.0 + z)
    z = np.exp(s)
    return z / (1.0 + z)

def bce_sum(y, e, eps=1e-12):
    y = np.clip(y, eps, 1.0 - eps)
    return float(-np.sum(e * np.log(y) + (1.0 - e) * np.log(1.0 - y)))

def mse_mean(y, e):
    diff = np.clip(e - y, -1e6, 1e6)
    return float(np.mean(diff * diff))

def sse_sum(y, e):
    y = np.asarray(y, dtype=float).reshape(-1)
```

```
    e = np.asarray(e, dtype=float).reshape(-1)
    return float(np.sum((y - e) ** 2))

def alpha_adaptive(x):
    return 1.0 / (1.0 + float(np.sum(x ** 2)))

class LR1Net:
    def __init__(self, lr=0.1, seed=42, w_clip=50.0):
        rng = np.random.default_rng(seed)
        self.w = rng.uniform(-0.5, 0.5, size=2).astype(float)
        self.b = float(rng.uniform(-0.5, 0.5))
        self.lr = float(lr)
        self.w_clip = float(w_clip)

    def forward_linear(self, x):
        return float(np.dot(self.w, x) + self.b)

    def train_epoch(self, X, E, shuffle=True, seed=0):
        idx = np.arange(len(X))
        if shuffle:
            rng = np.random.default_rng(seed)
            rng.shuffle(idx)
        for i in idx:
            x = X[i]
            e = E[i]
            y = self.forward_linear(x)
            err = e - y
            self.w += self.lr * err * x
            self.b += self.lr * err
            self.w = np.clip(self.w, -self.w_clip, self.w_clip)
            self.b = float(np.clip(self.b, -self.w_clip, self.w_clip))
        y_all = np.array([self.forward_linear(x) for x in X], dtype=float)
        return mse_mean(y_all, E)

def run_lr1_mse_fixed():
    x_scale = np.max(np.abs(X_raw), axis=0)
    X = X_raw / x_scale
    learning_rates = [0.01, 0.05, 0.1, 0.2, 0.5, 0.8]
    max_epochs = 100
    histories = {}
    best_lr = None
    best_final = float("inf")
    for lr in learning_rates:
        model = LR1Net(lr=lr, seed=42, w_clip=50.0)
        hist = []
        for ep in range(max_epochs):
            cur = model.train_epoch(X, E, shuffle=True, seed=1000 + ep)
            cur_es = cur * len(E)
```

```
            hist.append(cur_es)
            if cur_es < 1e-8 * len(E):
                break
        histories[lr] = np.array(hist, dtype=float)
        if hist[-1] < best_final:
            best_final = hist[-1]
            best_lr = lr
    model = LR1Net(lr=best_lr, seed=42, w_clip=50.0)
    for ep in range(len(histories[best_lr])):
        model.train_epoch(X, E, shuffle=True, seed=1000 + ep)
    return histories[best_lr], best_lr, x_scale, model.w.copy(), float(model.b)

class LR2Net:
    def __init__(self, seed=42, w_clip=50.0):
        rng = np.random.default_rng(seed)
        self.w = rng.uniform(-0.5, 0.5, size=(2,))
        self.T = 0.0
        self.w_clip = float(w_clip)

    def forward(self, x):
        return float(np.dot(self.w, x) - self.T)

    def update_delta_rule(self, x, e, alpha):
        y = self.forward(x)
        err = (y - e)
        self.w = self.w - alpha * err * x
        self.T = self.T + alpha * err
        self.w = np.clip(self.w, -self.w_clip, self.w_clip)
        self.T = float(np.clip(self.T, -self.w_clip, self.w_clip))

def run_lr2_mse_adaptive(alpha_fixed=0.1, Ee=1e-6, max_epochs=2000, seed=123):
    x_scale = float(np.max(np.abs(X_raw)))
    X = X_raw / x_scale
    model = LR2Net(seed=42, w_clip=50.0)
    rng = np.random.default_rng(seed)
    hist = []
    n = X.shape[0]
    for _ in range(max_epochs):
        idx = np.arange(n)
        rng.shuffle(idx)
        for i in idx:
            x = X[i]
            e = E[i]
            alpha = alpha_adaptive(x)
            model.update_delta_rule(x, e, alpha)
        y_all = np.array([model.forward(x) for x in X], dtype=float)
        Es = sse_sum(y_all, E)
        hist.append(Es)
```

```
        if Es <= Ee:
            break
    return np.array(hist, dtype=float)

class SigmoidNet:
    def __init__(self, seed=42, w_clip=50.0):
        rng = np.random.default_rng(seed)
        self.w = rng.uniform(-0.5, 0.5, size=(2,))
        self.T = 0.0
        self.w_clip = float(w_clip)

    def s(self, x):
        return float(np.dot(self.w, x) - self.T)

    def y(self, x):
        return float(sigmoid(self.s(x)))

    def update_bce(self, x, e, alpha):
        y = self.y(x)
        err = (y - e)
        self.w = self.w - alpha * err * x
        self.T = self.T + alpha * err
        self.w = np.clip(self.w, -self.w_clip, self.w_clip)
        self.T = float(np.clip(self.T, -self.w_clip, self.w_clip))

def train_bce(mode, alpha_fixed, Ee, max_epochs=5000, seed=123):
    x_scale = np.max(np.abs(X_raw), axis=0)
    X = X_raw / x_scale
    model = SigmoidNet(seed=42, w_clip=50.0)
    rng = np.random.default_rng(seed)
    hist = []
    n = X.shape[0]
    for _ in range(max_epochs):
        idx = np.arange(n)
        rng.shuffle(idx)
        for i in idx:
            x = X[i]
            e = float(E[i])
            alpha = float(alpha_fixed) if mode == "fixed" else alpha_adaptive(x)
            model.update_bce(x, e, alpha)
        y_all = np.array([model.y(x) for x in X], dtype=float)
        Es = bce_sum(y_all, E)
        hist.append(Es)
        if Es <= Ee:
            break
    return model, np.array(hist, dtype=float), x_scale

def boundary_raw_from_bce(model, x_scale_vec, x1_vals):
```

```
    w1, w2 = model.w
    T = model.T
    xs1, xs2 = x_scale_vec
    if abs(w2) < 1e-12:
        return None
    return xs2 * (T - w1 * (x1_vals / xs1)) / w2

def boundary_raw_from_lr1(best_w, best_b, x_scale_vec, x1_vals, threshold=0.5):
    a1 = best_w[0] / x_scale_vec[0]
    a2 = best_w[1] / x_scale_vec[1]
    if abs(a2) < 1e-12:
        return None
    return (threshold - best_b - a1 * x1_vals) / a2

def main():
    Ee = 1e-6
    alpha_best = 0.2

    hist_mse_fixed, best_lr, x_scale_lr1, w_lr1, b_lr1 = run_lr1_mse_fixed()
    hist_mse_adapt = run_lr2_mse_adaptive(alpha_fixed=0.1, Ee=Ee, max_epochs=2000)

    model_bce_fixed, hist_bce_fixed, x_scale_bce = train_bce("fixed", alpha_best, Ee)
    model_bce_adapt, hist_bce_adapt, _ = train_bce("adaptive", alpha_best, Ee)

    labels = [
        f"MSE + фикс (lr={best_lr})",
        "MSE + адапт",
        "BCE + фикс ",
        "BCE + адапт "
    ]
    hists = [hist_mse_fixed, hist_mse_adapt, hist_bce_fixed, hist_bce_adapt]

    plt.figure(figsize=(10, 6))
    for h, lbl in zip(hists, labels):
        plt.plot(np.arange(1, len(h) + 1), h, label=lbl, linewidth=1.8)
    plt.xlabel("Эпоха p")
    plt.ylabel("Es(p)")
    plt.title("Сходимость Es(p) для 4 конфигураций")
    plt.grid(True, alpha=0.35)
    plt.legend()
    plt.tight_layout()
    plt.show()

    fig, ax = plt.subplots(figsize=(9, 7))
    X0 = X_raw[E == 0]
    X1 = X_raw[E == 1]
    ax.scatter(X0[:, 0], X0[:, 1], s=70, alpha=0.8, label="Класс 0")
    ax.scatter(X1[:, 0], X1[:, 1], s=70, alpha=0.8, label="Класс 1")
```

```
x1_min, x1_max = X_raw[:, 0].min() - 2, X_raw[:, 0].max() + 2
x1_vals = np.linspace(x1_min, x1_max, 200)

x2_mse = boundary_raw_from_lr1(w_lr1, b_lr1, x_scale_lr1, x1_vals, threshold=0.5)
if x2_mse is not None:
    ax.plot(x1_vals, x2_mse, linewidth=2.2, label="Линия MSE")

x2_bce = boundary_raw_from_bce(model_bce_adapt, x_scale_bce, x1_vals)
if x2_bce is not None:
    ax.plot(x1_vals, x2_bce, linewidth=2.2, linestyle="--", label="BCE (адапт. шаг)")

ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Точки и разделяющие линии")
ax.grid(True, alpha=0.35)
ax.legend()
plt.tight_layout()
plt.show()

print(f"\n{'Конфигурация':<30} {'Эпох':>6} {'Финал Es':>14}")
print("-" * 52)
for name, hist in zip(labels, hists):
    print(f"{name:<30} {len(hist):>6} {hist[-1]:>14.6e}")

threshold = 0.5
user_points = []
user_probs = []

while True:
    s = input("\nx1 x2 > ").strip()
    if s.lower() in ("q", "quit", "exit"):
        break
    try:
        x1s, x2s = s.replace(",", " ").split()
        x_user_raw = np.array([float(x1s), float(x2s)], dtype=float)
    except Exception:
        print("Ошыбка: введи два числа через пробел или q")
        continue

    x_user = x_user_raw / x_scale_bce
    p = model_bce_adapt.y(x_user)
    cls = 1 if p >= threshold else 0
    print(f"y={p:.6f} class={cls}")

    user_points.append([x_user_raw[0], x_user_raw[1]])
    user_probs.append(p)
```

```
    fig, ax = plt.subplots(figsize=(9, 7))
    ax.scatter(X0[:, 0], X0[:, 1], s=70, alpha=0.8, label="Класс 0")
    ax.scatter(X1[:, 0], X1[:, 1], s=70, alpha=0.8, label="Класс 1")
    up = np.array(user_points, dtype=float)
    ax.scatter(up[:, 0], up[:, 1], marker="x", s=140, linewidths=2.5, label="Введённые
точки")
    for (x1p, x2p), pp in zip(user_points, user_probs):
        ax.text(x1p + 0.1, x2p + 0.1, f"y={pp:.3f}", fontsize=9)

    x2_mse = boundary_raw_from_lr1(w_lr1, b_lr1, x_scale_lr1, x1_vals, threshold=0.5)
    if x2_mse is not None:
        ax.plot(x1_vals, x2_mse, linewidth=2.2, label="Линия MSE")
    x2_bce = boundary_raw_from_bce(model_bce_adapt, x_scale_bce, x1_vals)
    if x2_bce is not None:
        ax.plot(x1_vals, x2_bce, linewidth=2.2, linestyle="--", label="BCE (адапт. шаг)")

    ax.set_xlabel("x1")
    ax.set_ylabel("x2")
    ax.set_title("Режым функцианирования")
    ax.grid(True, alpha=0.35)
    ax.legend()
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    main()
```

Результат:

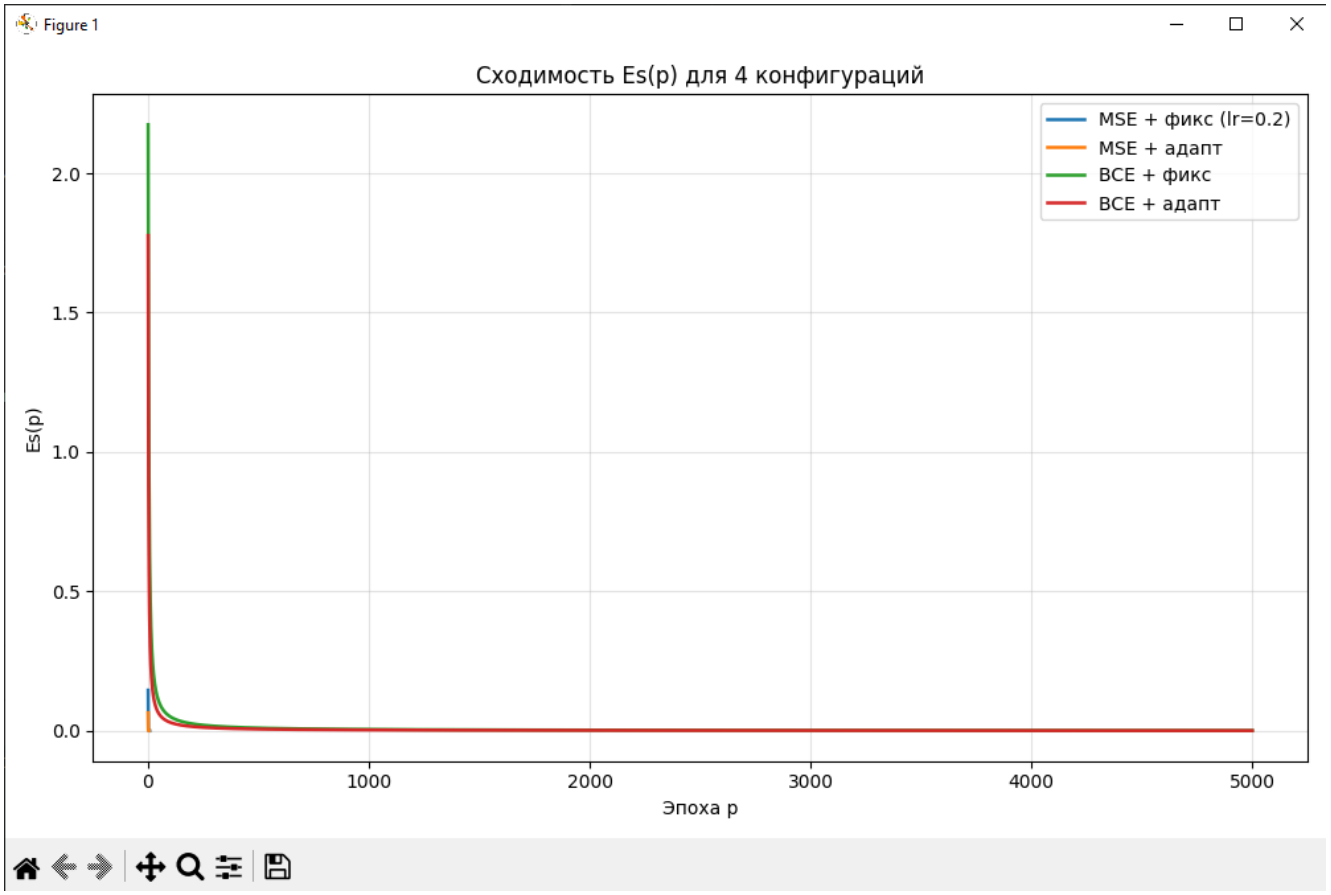
```
C:\Users\сеня\Desktop\.venv\Scripts\python.exe C:\Users\сеня\Desktop\.venv\giislab1.py

Конфигурация                Эпох      Финал Es
-----------------------------------------------
MSE + фикс (lr=0.2)           8   4.268176e-09
MSE + адапт                   5   3.108042e-08
BCE + фикс                 5000   1.001147e-03
BCE + адапт                5000   6.002964e-04

x1 x2 > 3 3
y=0.998646 class=1

x1 x2 > 1 1
y=0.900348 class=1

x1 x2 > -3 -3
y=0.001354 class=0

x1 x2 > -2 -2
y=0.012102 class=0
```

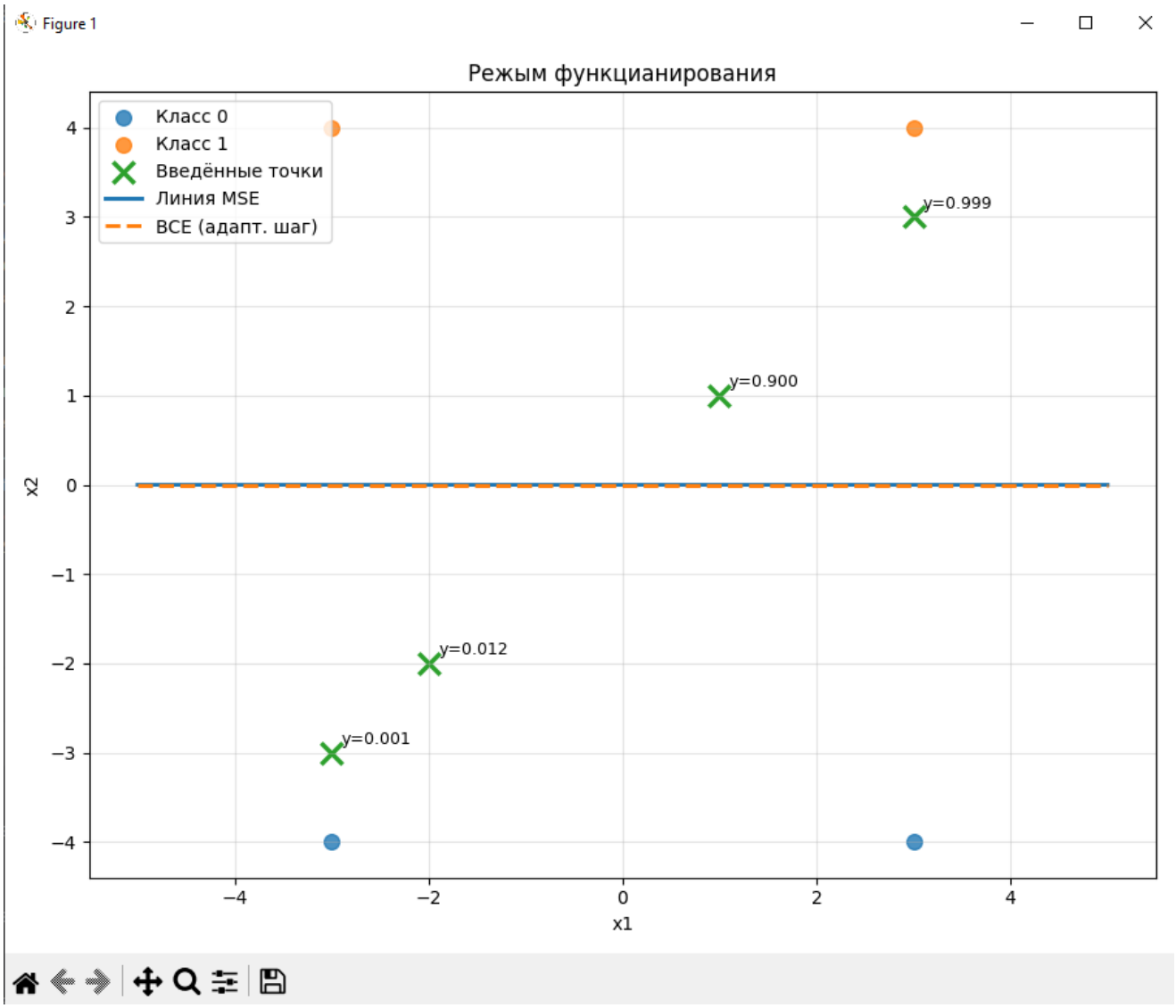
Анализ результатов

В результате экспериментов установлено, что модель с MSE сходится быстрее по числу эпох, так как минимизирует квадратичную ошибку напрямую для линейной модели. Модель с BCE использует сигмоиду и имеет вероятностную интерпретацию, поэтому уменьшение ошибки происходит медленнее и носит асимптотический характер. Адаптивный шаг делает обучение более устойчивым и стабилизирует процесс сходимости.

Разделяющие линии, полученные разными методами, практически совпадают, поскольку данные линейно разделимы. Различия связаны с разными функциями потерь.

Рисунки с результатами работы программы





Вывод:Изучил принципы бинарной классификации с вероятностной точки зрения. Реализовал однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провел **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).